Center Name: ACE-HCMC-2-FPT.

MEDIFLOW

Developer's Guide

Supervisor:	Mr. HOANG DUC QUANG	
Semester:	4	
Batch No:	T2.2410.E0	
Group No:	4	
Order:	**Full name**	**Roll No.**
1.	Pham Dang Vinh	Student1614910
2.	Tran Hoang Anh	Student1614884
3.	Than The Loc	Student1571994
4.	Le Quang Huy	Student1394600

- **This is to certify that**

Mr. Pham Dang Vinh

Mr. Tran Hoang Anh

Mr. Than The Loc

Mr. Le Quang Huy

have successfully designed & developed:

eProject: MediFlow

- **Submitted by:**

Mr. Hoang Duc Quang

Date of issue: 08/11/2026

Authorized Signature: _____

Table of Contents

Table of Contents 2
I. Download Source Code (BE, FE & Mobile) 1
II. Install the Prerequisites 2
III. Configure the Environment 2
IV. Run the Project (Demo Data Auto-Import) 5
V. Run the Project Manually (Docker Compose & Maven) 9
VI. Run the Mobile App (Flutter) 12
VII. Architecture and Source Code Structure 13
VIII. Testing and Quality Checks 15
IX. Stop, Clean Up and Troubleshooting 16

I. Download Source Code (BE, FE & Mobile)

MediFlow is a single Git monorepo that contains the Spring Boot backend, the Next.js web client and the Flutter mobile client.

Step 1. Clone the repository (or extract the submitted source archive):

```
git clone git@github.com:Dangvinh77/MediFlow.git

cd MediFlow
```

Step 2. Open the **backend** in IntelliJ IDEA: **File → Open →** select the root `pom.xml` and choose **Open as Project**. IntelliJ imports all Maven modules of the reactor.

Step 3. Open the **frontend** folder `frontend/` in Visual Studio Code.

Step 4. Open the **mobile** folder `mobile/` in Visual Studio Code or Android Studio (Flutter plugin installed).

Step 5. On a new clone, run the bootstrap once. It installs the Git hooks that enforce the commit-author policy:

```
powershell -ExecutionPolicy Bypass -File

scripts/bootstrap.ps1
```

Repository layout

Folder	Content
backend/	13 Maven modules: eureka-server, gateway, common and the business services
frontend/	Next.js 16 web client (App Router, TypeScript, Tailwind CSS v4, pnpm)
mobile/	Flutter mobile client (Clean Architecture, Riverpod)
docs/	Design documents, API contracts, coding standards (docs/ai) and acceptance evidence
scripts/	Bootstrap, Git hooks, database init script and the Windows demo launcher (scripts/run)

Folder	Content
docker-compose.yml	PostgreSQL, RabbitMQ and the containerised services
pom.xml	Root Maven reactor

II. Install the Prerequisites

Install the tools below and make sure each command is available in the **PATH** (open a new terminal after installing).

Tool	Version	Used for	Check command
JDK	21	Build and run the backend	`java -version`
Apache Maven	3.9+	Backend build and tests	`mvn -v`
Docker Desktop	Latest	PostgreSQL, RabbitMQ, Testcontainers and the demo launcher	`docker info`
Python	3.x	Demo launcher (database restore and seeding)	`python --version`
Node.js + pnpm	Node 20+	Web client	`node -v`, `pnpm -v`
Flutter	Stable (Dart ^3.13)	Mobile client	`flutter doctor`

> **Tip:** Docker Desktop must be started and show **Engine running** before any backend command. Every bounded context owns its own PostgreSQL database inside Docker.

III. Configure the Environment

1. Email (SMTP) settings — smtp.local.env

MediFlow sends welcome emails, temporary passwords and password-reset links. SMTP settings are read from a **gitignored** file `smtp.local.env` in the repository root.

Step 1. Copy the template `smtp.env.example` to `smtp.local.env` in the project root.

Step 2. Fill in your Gmail sender and a Google **App Password** (not your normal password):

```
SPRING_MAIL_HOST=smtp.gmail.com
SPRING_MAIL_PORT=587
SPRING_MAIL_USERNAME=<your-email>
SPRING_MAIL_PASSWORD=<your-google-app-password>
MEDIFLOW_MAIL_FROM=<your-email>
MEDIFLOW_MAIL_FROM_NAME=MediFlow
MEDIFLOW_PUBLIC_WEB_URL=http://localhost:3000
MEDIFLOW_SECURITY_MAIL_KEY=<base64-encoded-32-byte-key>
MEDIFLOW_DEMO_MAILBOX_ENABLED=false
```

Step 3. Generate the mail key once, for example in PowerShell:

```
[Convert]::ToBase64String((1..32 | ForEach-Object { Get-Random -Maximum 256 }))
```

Step 4. Save the file. `scripts/run/start-backend.bat` loads it automatically.

For IDE/Java startup, dot-source `scripts/load-smtp-local.ps1` first.

> Never commit `smtp.local.env` or put real credentials in `smtp.env.example`. Without SMTP the system still works; emails stay pending and the local **demo mailbox** (`/demo-mailbox`) can be used for seed accounts.

2. *Backend variables (manual run only)*

When services are started from the IDE or with `java -jar`, provide these environment variables (or a gitignored `application-local.yml`). The demo launcher sets them for you.

Variable	Meaning	Local example
MEDIFLOW_DB_USER / MEDIFLOW_DB_PASSWORD	PostgreSQL account used by every service	postgres / postgres
MEDIFLOW_RABBIT_HOST / _PORT	RabbitMQ broker	localhost / 5672
MEDIFLOW_RABBIT_USER / _PASSWORD	RabbitMQ account	guest / guest
MEDIFLOW_JWT_SECRET	Shared JWT signing secret — must be identical for Gateway and Core	a long random string
MEDIFLOW_PUBLIC_WEB_URL	Web origin used in email links	http://localhost:3000

3. *Frontend variables — frontend/.env.local*

Copy `frontend/.env.example` to `frontend/.env.local`. The web client only calls the Gateway through the same-origin `/api/*` proxy.

```
GATEWAY_URL=http://localhost:8080

MEDIFLOW_OUTPATIENT_UI_ENABLED=true
```

`start-frontend.bat` writes the right values automatically when the demo launcher is used (Gateway port `28080`).

IV. Run the Project (Demo Data Auto-Import)

The fastest way on Windows is the demo launcher in `scripts/run/`. It builds the backend, creates the PostgreSQL/RabbitMQ containers, **restores the synthetic demo database automatically** (departments, staff, accounts, patients, schedules, drugs, prices) and starts all eight applications. You do not need to import any data manually.

1. *Start the backend*

Step 1. Open **Docker Desktop** and wait until the engine is running.

Step 2. Double-click `scripts\run\start-backend.bat`, or run from a terminal:

```
cd scripts\run

.\start-backend.bat
```

Run it **without** the `-Fast` option. The launcher then builds every backend module from source and imports the demo data from scratch, so the result does not depend on an earlier build or on the Docker state. Add `-LocalOnly` to keep everything on 127.0.0.1, or `-Rebuild` to force a clean Maven build.

Step 3. Wait for the console to print **CALENDAR READY** and then **READY**. Building from scratch takes several minutes (Maven build + data import). Keep the window open; the applications stop after two hours or when you press **Ctrl + C**.

```
Building the eight demo applications...

Gateway: http://127.0.0.1:28080.

eureka-server UP

...

CALENDAR READY

READY
```

Option	Effect
(none) — recommended	Build from source and import fresh demo data; applications stop after two hours
-Rebuild	Force a Maven rebuild
-ResetData	Recreate the retained demo data
-LanAddress <ip>	Serve on that LAN address so phones on the same Wi-Fi can connect
-LocalOnly	Bind to 127.0.0.1 only
-StartupParallelism 1	Start applications one at a time on low-memory machines

2. *Start the frontend*

Step 1. After the backend prints **READY**, double-click `scripts\run\start-frontend.bat` (it runs `pnpm install` and `pnpm dev`).

```
▲ Next.js 16.2.11 (Turbopack)

- Local:   http://127.0.0.1:3000

✓ Ready
```

Step 2. Press **Ctrl + Click** on the printed link, or open **http://127.0.0.1:3000/login** in the browser.

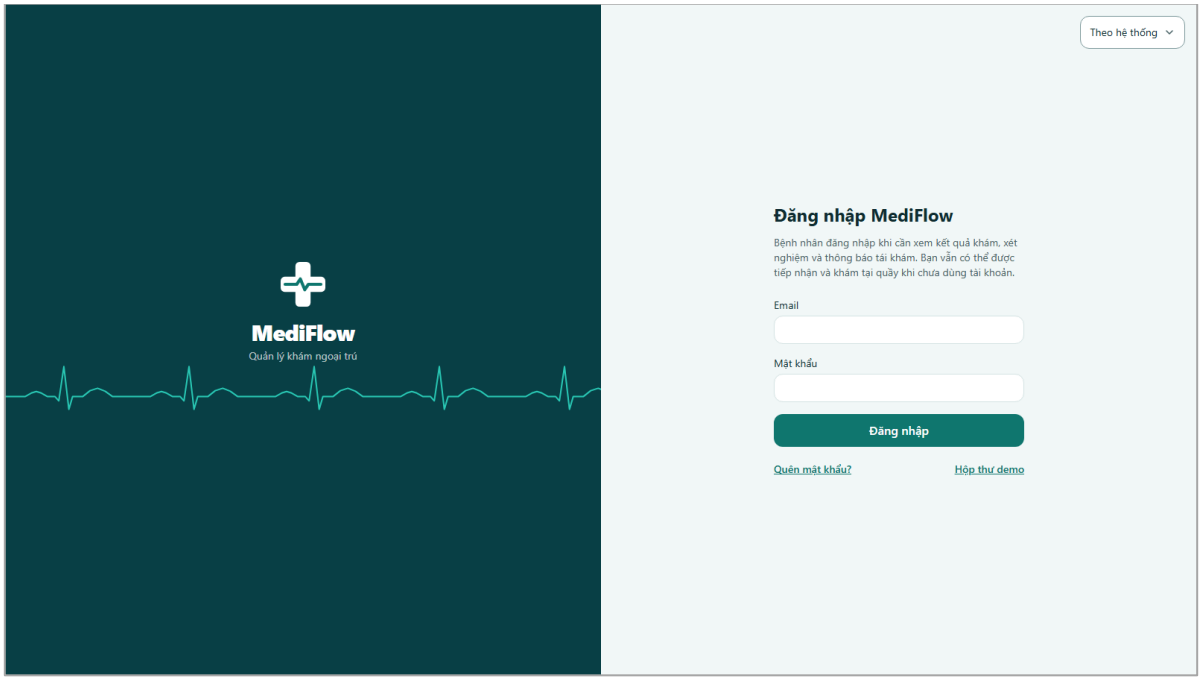


Figure 1. MediFlow login page served by the web client

Step 3. Sign in with a demo account below. If everything is set up correctly, the role workspace opens.

3. *Demo accounts*

All seed accounts share the password **MediFlowDemo@2026** (synthetic data only). Sign in with the **email**.

Role	Email	Name / department	Main use
ADMIN	demo.staff01@example.test	Nguyễn Quản Trị Demo	Organization, staff, accounts, department heads, reports

Role	Email	Name / department	Main use
NURSE	demo.staff04@example.test	Phạm Điều Dưỡng Demo — Internal Medicine	Patient records, patient accounts, reception
DOCTOR	demo.staff03@example.test	Lê Bác Sĩ Demo — Internal Medicine (department head)	Examination, lab orders, prescriptions, department management
DOCTOR	demo.staff05@example.test	Nguyễn Minh Phúc Demo — Paediatrics	Second doctor, another department
LAB_TECH	demo.staff15@example.test	Dương Thanh Tâm Demo — Laboratory	Run tests and enter results
PHARMACIST	demo.staff17@example.test	Mai Thảo Vy Demo — Pharmacy	Drug catalogue, dispensing
CASHIER	demo.staff19@example.test	Phan Khánh Linh Demo — Finance	Exam, lab and prescription payments
PATIENT	demo.patient01@example.test	Nguyễn Văn An Demo	Own results and notifications
PATIENT	demo.patient02@example.test	Trần Thị Bình Demo	Follow-up patient

> The complete list (with patient IDs and scenarios) is in `scripts/run/TAI_KHOAN_DEMO.txt`; the step-by-step demo script is `scripts/demo_25_10/DEMO_UI_TUNG_BUOC.md`.

V. Run the Project Manually (Docker Compose & Maven)

Use this path on macOS/Linux or when you want to debug individual services in the IDE.

Step 1. Start the shared infrastructure. On first start PostgreSQL runs `scripts/init-databases.sql`, which creates one database per context:

```
docker compose up -d postgres rabbitmq
```

Step 2. Build all backend modules:

```
mvn -q -DskipTests install
```

Step 3. Start the applications in this order (each in its own terminal or IntelliJ run configuration). Flyway creates the schema of each database on startup:

```
java -jar backend/eureka-server/target/eureka-server-0.0.1-SNAPSHOT.jar

java -jar backend/gateway/target/gateway-0.0.1-SNAPSHOT.jar

java -jar backend/core-service/target/core-service-0.0.1-SNAPSHOT.jar

java -jar backend/clinical-service/target/clinical-service-0.0.1-SNAPSHOT.jar

java -jar backend/lab-service/target/lab-service-0.0.1-SNAPSHOT.jar

java -jar backend/pharmacy-service/target/pharmacy-service-0.0.1-SNAPSHOT.jar

java -jar backend/billing-service/target/billing-service-0.0.1-SNAPSHOT.jar

java -jar backend/support-service/target/support-service-0.0.1-SNAPSHOT.jar
```

Step 4. Check the services in the Eureka dashboard **http://localhost:8761** and the Gateway health endpoint **http://localhost:8080/actuator/health**.

Step 5. Start the web client:

```
cd frontend

pnpm install

pnpm dev
```

Step 6. Open **http://localhost:3000**. `pnpm dev` proxies `/api/*` to the Gateway on port 8080.

Module	Port	Role
eureka-server	8761	Service discovery
gateway	8080	/api/* entry point, login and route authorization
core-service	8089	Organization and Patient contexts (/api/v1/org, /api/v1/patients)
clinical-service	8082	Appointments and medical records (/api/v1/appointments, /api/v1/records)
lab-service	8084	Laboratory (/api/v1/lab)
pharmacy-service	8085	Pharmacy (/api/v1/pharmacy)
billing-service	8086	Billing (/api/v1/billing)
support-service	8087	Report and Notification (/api/v1/reports, /api/v1/notifications)
PostgreSQL / RabbitMQ	5432 / 5672 (UI 15672)	Databases per context / event broker

> There is no default administrator in a clean manual setup. Create accounts through a seeded environment, or use the demo launcher which provisions all demo accounts.

VI. Run the Mobile App (Flutter)

The Flutter client talks to the real Gateway only (no mock data). From the `mobile/` folder on Windows:

Step 1. Fetch packages and run the web build with a same-origin proxy to the demo Gateway:

```
cd mobile

flutter pub get

./tool/run_web.ps1 -Demo -Gateway http://127.0.0.1:28080
```

Step 2. Open **http://localhost:4300**, then use the browser device toolbar (F12 → Ctrl+Shift+M) to emulate a phone.

> If Windows PowerShell 5.1 stops the script right after the message "Wasm dry run succeeded", the web build has already finished: start the server directly with `dart run tool/dev_server.dart 4300 http://127.0.0.1:28080`. The first page load can take about ten seconds.

Step 3. For hot reload on Chrome or an Android emulator:

```
./tool/run_dev.ps1

./tool/run_dev.ps1 -Target android -Demo -Gateway
http://127.0.0.1:28080
```

Step 4. For a real device on the same Wi-Fi, start the backend with `-LanAddress <your-ip>` and run:

```
flutter run -d <device-id> --dart-
define=GATEWAY_URL=http://<your-ip>:28080/api \
  --dart-define=ALLOW_INSECURE_HTTP=true --dart-
define=MEDIFLOW_OUTPATIENT_UI_ENABLED=true \
  --dart-define=PHARMACY_MODE=outpatient
```

> `GATEWAY_URL` must end with `/api`. An Android emulator reaches the host machine through `10.0.2.2`. Never point the app at a service port directly.

VII. Architecture and Source Code Structure

MediFlow is a microservices-based outpatient hospital information system. Clients call one **Spring Cloud Gateway**; services register in **Eureka**, own a separate **PostgreSQL** database and exchange domain events through **RabbitMQ** (exchange `mediflow.events`). Contexts reference each other only by UUID — there is no cross-context database access.

1. *Backend service layout (clean architecture)*

```
src/main/java/com/mediflow/<context>/
├── domain/          pure Java model and business rules (no
Spring/JPA)
├── application/     use cases with port/in and port/out
└── infrastructure/  web, persistence, messaging, clients,
security, config
```

Dependencies point inward only: `infrastructure → application → domain`. Every endpoint declares roles with `@PreAuthorize`; the Gateway also checks routes. Money uses `BigDecimal`, identifiers `UUID`, dates `LocalDate`/`Instant`.

2. ***Web client layout***

```
frontend/src/
├── app/          routes (App Router): login, appointments,
records, lab, ...
├── features/     one folder per bounded context:
│                 appointment, billing, lab, medical-record,
notification,
│                 organization, patient, pharmacy, report
├── components/   shared UI components
└── lib/          api.ts (the only HTTP client), shared types
```

Main business events

Action in UI	Service	Event published
Create / update patient	Core (Patient)	patient.created / patient.updated
Register visit and hold a slot	Clinical	outpatient.registration.changed
Pay a charge	Billing	payment.completed, financial.clearance.granted
Add diagnosis / complete record	Clinical	diagnosis.added / medicalrecord.completed

Action in UI	Service	Event published
Create lab request / save result	Lab	lab.request.created / lab.result.created
Create prescription / dispense	Pharmacy	prescription.created / prescription.filled
Store in-app notification	Support (Notification)	notification.sent

VIII. Testing and Quality Checks

Part	Command	What it runs
Backend (all)	`mvn test`	Unit, web-slice, persistence and Testcontainers integration tests (Docker required)
Backend (one module)	`mvn -pl backend/core-service -am test`	Tests of one service and its dependencies
Web client	`pnpm typecheck`, `pnpm lint`, `pnpm test`	TypeScript, ESLint and Vitest
Mobile client	`./tool/check.ps1`	Format, analyze, tests and a web build

IX. Stop, Clean Up and Troubleshooting

Step 1. Stop the frontend: press **Ctrl + C** in its window.

Step 2. Stop the backend with **Ctrl + C** in the backend window. The next `start-backend.bat` builds the modules and imports the demo data again from scratch.

Step 3. Preview, then remove the project containers and images (volumes are kept):

```
.\clean-docker.bat -Preview

.\clean-docker.bat
```

Problem	Solution
Launcher fails with a Docker error	Start Docker Desktop and wait for the engine, then run the launcher again
Login says the credentials are wrong	Use the email (not the old username) and the demo password; a password changed during the session is kept
A page returns 404 right after a build	Stop only the frontend, move frontend/.next/dev aside and run start-frontend.bat again
Phone cannot reach the Gateway	Start with -LanAddress <ip> (run once as administrator to add the firewall rule) and open http://<ip>:28080/actuator/health on the phone
Low memory during startup	Use -StartupParallelism 1
Emails are not received	Check smtp.local.env and the Google App Password; seed accounts can use /demo-mailbox